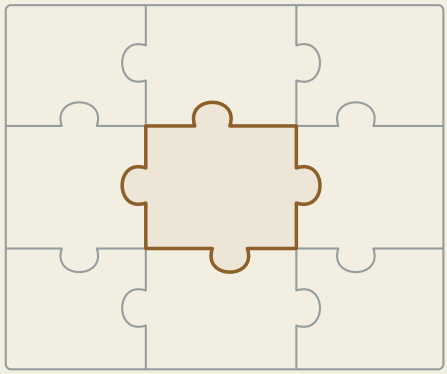
Scaffold Kit

A plain-language guide

Turn an idea into a clear plan, then build one useful piece at a time.



> Plan broadly. Build narrowly.
> Test reality. Expand deliberately.

A skill is a set of instructions your AI assistant can follow. Scaffold Kit helps you explain what you want, compare important choices, and agree on what the assistant may build next.

For the 0.4.4 stable release.
Updated October 17, 2026.

Keep the plan and the permission clear

A long conversation can leave different people imagining different products. Important choices disappear into chat, and a small change can grow into a much larger build.

The kit saves the plan inside the project. It keeps the wider design visible while one approved piece defines the current work. That piece is called a **slice**: a useful build increment with a clear boundary and checks.

Plan	Choose	Approve	Build	Check
Understand the whole project	Bound the next useful slice	Review that specific brief	Stay inside the approved scope	Compare results with intent

The owner stays in charge

Discussing an idea, accepting a recommendation, approving a build, and checking its result are separate actions. The assistant must keep them separate too.

This field brief explains the kit. A project's Slice Brief is a different document: it defines the next build and carries that build's approval.

One plan, with a home for each kind of information

The documents live with the project, usually in its `docs/` folder. They link to each other so reasons and evidence do not need to be copied into every page.

`ARCHITECTURE.md`

The whole puzzle

What are we making? Who uses it? How do its main parts fit together, and where can later pieces connect?

`ENGINEERING.md`

The rules for building

What must it do? How will it handle data and risk? What checks and standards define acceptable work?

`DECISION-LOG.md`

The choices and their reasons

What did we choose, what else did we consider, and what would make us reconsider? Unanswered choices stay visible.

`SLICE-NNN-name.md`

The next build boundary

What can be built now? What stays outside this slice? Which files, effects and observable checks are included?

A small brief can still carry a firm boundary

Small features and contained fixes normally use 250 to 400 words. The compact form caps the whole brief at 500, including its status block and tables, unless a named risk needs essential decision detail.

A recipe box, as an example

A possible first slice is to **save and reopen one recipe**.

The wider plan can reserve room for **sharing or meal planning later**. Their exclusion and connection points belong in the brief; they do not authorize building those features now.

If the owner has not chosen where recipes live or how lost changes are recovered, those choices **remain pending**. The assistant must not invent a storage service, a backup promise, or fake substitute features to fill the gap.

Labels help trace the plan: R = requirement, D = decision, S = acceptance check, A = assumption, U = unknown. Each points to its owning record. These are traceability labels, not extra features to build.

The recipe example is illustrative. It is not an approved project or an evaluation result.

Ask what matters, then save the answers

1. **Describe the project**

   Say who it is for, the result you want, and what already exists. Rough descriptions are welcome. Prior research can inform the plan; it does not count as permission.

2. **Choose the explanation depth**

   Pick plain language, balanced, or developer shorthand. The choice is saved in the project's Triage Card, a short record of its size, risks and starting point. Depth changes the wording, not the evidence or approval rules.

3. **Compare the consequential choices**

   The assistant presents realistic options, their practical tradeoffs, and a recommendation. You may choose, explicitly delegate an ordinary choice, or leave it unresolved. Silence is never consent.

4. **Review one slice and its checks**

   The brief names included work, exclusions, files, possible side effects, blockers and checks for success. Risky unknowns need a resolving check before work depends on their answer.

5. **Approve the specific build**

   Approval identifies the brief's version, targets and effects, with your name and date. Liking the idea or approving an earlier slice does not approve this one.

> ### Less discussion does not remove the checkpoints
>
> Fast-run presents defaults for explicit review. A request to draft without follow-up produces a provisional plan with unanswered choices and approval still pending.

Choices involving payment providers, where data is stored, connections other programs rely on, or deleting user data need explicit human confirmation. The assistant must explain why they are difficult to undo.

For existing software, establish what the relevant code actually does first. For the next slice, reuse the current documents and ask about the gaps; do not restart the whole interview.

A clear plan is a starting point

The kit preserves the distinction between a choice, permission to act, and evidence that the action worked.

Decision

`Confirmed` means the human chose or explicitly delegated it. `Proposed` is a recommendation; `Assumed` is unverified; `Open` is unresolved. `Deferred` waits for its revisit trigger; `Superseded` links to its replacement.

Build

The current brief must record

```
Approved for build by: <name>, <date>
```

and its version. The builder follows that boundary. Changed scope or new effects reopen the affected approval; routine evidence notes do not.

Verification

A planned check has not run. A configured command is not a passing result. Record what was executed, its result and limits, and the evidence location. Failed or skipped checks stay pending until relevant new evidence passes.

Closeout

Compare the actual changes and observed behavior with approved intent. Update affected records, retain cited evidence, and select the next slice. One slice stays in flight at a time.

> ### The document audit has a limited job
>
> The optional audit script checks document structure, identifiers, references, status rules and writing hygiene. A clean result does not grant approval, prove the software works, or establish that a person understands the explanation.

If reality disagrees with the plan

Report the contradiction and its source. Pause dependent work, preserve the recorded blocker, and propose the smallest correction through the existing Decision Log. Do not invent a plausible reason to make inconsistent records look complete.

Time spent checking work must serve a requirement and match its consequences. Building new testing tools needs its own justification; it is not an automatic addition to every small change.

Start with a project, not a stack of forms

Once Scaffold Kit is installed in your assistant, ask for a plan and state the boundary you want. For example:

> Use Scaffold Kit to plan a small recipe box for our household. We are new to building software, so use plain language. Start with the questions you need. Write planning documents only; do not build product code.

This is a starter prompt, not a benchmark or a guarantee about a generated answer.

What to expect back

- Questions about the people, desired outcome, risks and constraints.
- Important choices with practical alternatives and unanswered questions clearly marked.
- The four linked documents and one proposed Slice Brief for review.
- A build gate stated plainly, with decisions and approval still pending where needed.

How Anti-Dark-Code fits

Scaffold Kit records intended scope and reasons. Anti-Dark-Code examines existing code, findings and verification evidence. Use its relevant mapping or verification work when needed, and link to that evidence instead of copying it into a second ledger.

Their uses of "slice" differ: Scaffold slices are build increments; Anti-Dark-Code slices group code for coverage. Their statuses stay separate.

Use the smallest route that fits

A new product uses the planning interview.

An existing system starts by mapping the relevant code and unanswered questions.

A later feature reuses the existing plan.

A genuine throwaway experiment that answers one question and is then deleted may not need the kit.

Compatibility has limits

Local Claude Code and Codex host checks have been exercised, with corrections and limits retained in the evidence. That does not establish every platform's compatibility.

Source and installation guidance · FSL-1.1-MIT license

The source protocols and templates govern use; this brief is an explanation, not a new approval layer.